

Week12 예습과제

⚙ Status	Not started
≡ Text	373~424

Transformer

주요 기능: 병렬로 입력 시퀀스 처리 (긴 시퀀스를 순환 신경망 모델보다 빠르고 효율적으로 처리)

- 셀프 어텐션(Self-Attention) 기반
→ 재귀/합성곱 연산 없이 입력 토큰 간의 관계 직접 모델링 가능
- 대용량 데이터셋 학습 효율적
- 기계 번역, 언어 모델링, 텍스트 분류 효과적
- 자연어 처리 분야 영향력 큼

오토 인코딩

- 랜덤하게 문장의 일부를 빈칸 토큰으로 만들 → 해당 빈칸에 어떤 단어가 적절한지 예측하는 Task
- 인코더: 예측되는 토큰의 양옆에 있는 토큰 참조

트랜스포머 구조

A) 양방향 구조

트랜스포머 모델은 성능이 **높고** 다양한 구조로 활용된다

B) 단방향 구조

트랜스포머 모델은 성능이 **높고** ...

그림 7.1 트랜스포머 구조

디코더

- 예측되는 토큰의 왼쪽에 있는 토큰들만 참조 → 단방향 구조(B)

트랜스포머 모델 구조

표 7.1 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

- ELECTRA: 인코더에 생성적 적대 신경망 판별기 활용
- T5, BART: 자연어 처리 작업에 대한 학습

Transformer

- 딥러닝 모델 중 하나
- 기계 번역, 챗봇, 음성 인식 등 자연어 처리 분야에서 성과
- 어텐션 메커니즘만을 사용 → 시퀀스 임베딩 표현

어텐션 메커니즘

- 인코더-디코더 상호작용 → 입력 시퀀스의 중요한 부분에 초점 맞춰 문맥 이해&적절한 출력 생성
- 인코더: 입력 시퀀스 임베딩 → 고차원 벡터 변환
- 디코더: 인코더의 출력을 입력으로 받아 출력 시퀀스 생성
- 어텐션 메커니즘: 인코더-디코더 단어 사이의 상관관계 계산 → 중요한 정보에 집중
⇒ 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련 있는지 파악

장점

- 기존 순환 신경망 기반 모델보다 학습 속도 빠름
- 병렬 처리 가능 → 대규모 데이터셋에서 높은 성능
- 임베딩 과정에서 문장의 전체 정보 고려 → 긴 문장도 성능 유지

트랜스포머 모델구조

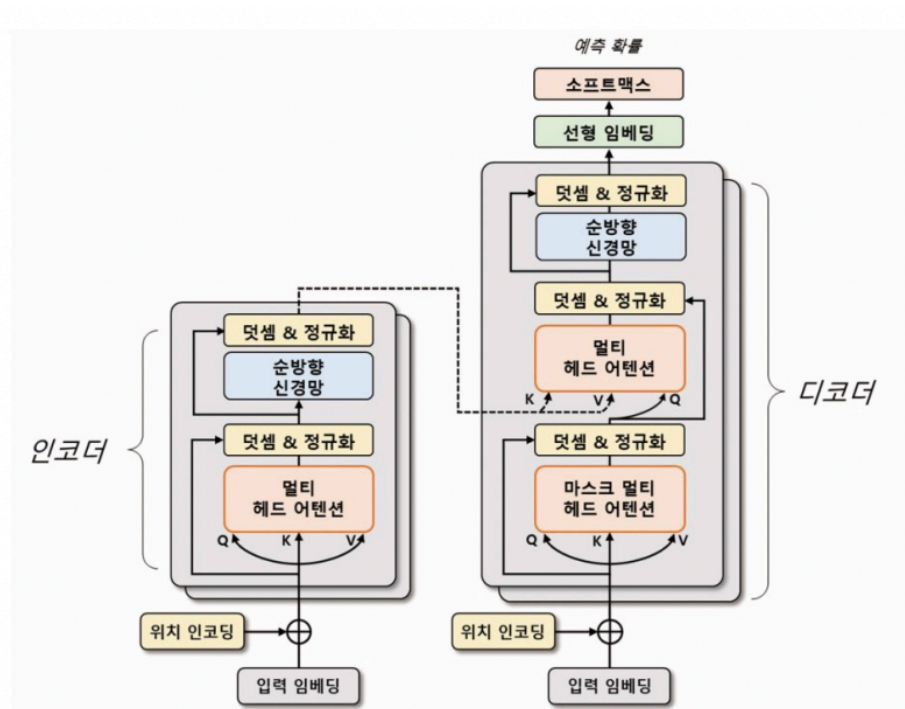


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

- 인코더와 디코더 각각 여러개의 트랜스포머 블록
 - 블록: 멀티 헤드 어텐션 + 순방향 신경망
 - 멀티 헤드 어텐션: 입력 시퀀스에서 Q, K, V 벡터 만들어 Self-Attention 수행
 - Q-K 유사도 계산 → Value에 가중합 → 단어 간 관계 반영한 새로운 임베딩 생성
- 순방향 신경망: 어텐션 결과 임베딩을 비선형 변환하여 의미를 더 풍부하게 만드는 역할
- 인코더 역할: 소스 시퀀스를 위치 인코딩과 함께 입력 → 관계 반영 벡터 출력
- 디코더 역할: 마스크드 멀티 헤드 어텐션 사용 → 타겟 시퀀스 순차적 생성
 - 인코더 출력도 참고 → 더 정확한 생성

- 최종 출력: 디코더의 마지막 출력 벡터 → 선형 변환 → 모델 출력으로 사용

입력 임베딩과 위치 인코딩

입력 임베딩

- 트랜스포머는 각 단어를 벡터 형태로 바꾸기 위해 임베딩 사용
- but RNN처럼 순서를 자연스럽게 인식 X → 단어 순서 정보가 사라짐

위치 임베딩

- 필요한 이유: 트랜스포머 병렬 입력 처리 → 단어가 문장에서 몇 번째인지 알 수 X
→ 단어 의미 벡터에 위치 정보 추가해서 순서 반영해야 함
- 위치를 나타내는 벡터 생성 → 임베딩 벡터에 더함 → 최종 입력 벡터
- 위치 벡터: sin함수 & cos 함수
 - 서로 다른 주기 sin/cos 사용 → 모델이 다양한 방식으로 상대적/절대적 위치 파악 가능
 - 각 위치마다 패턴이 규칙적, 유일한 값
 - 위치가 변해도 모델이 순서 정보 학습 가능

위치 인코딩

```
import math
import torch
from torch import nn
from matplotlib import pyplot as plt

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )
```

```

pe = torch.zeros(max_len, 1, d_model)
pe[:, 0, 0::2] = torch.sin(position * div_term)
pe[:, 0, 1::2] = torch.cos(position * div_term)
self.register_buffer("pe", pe)

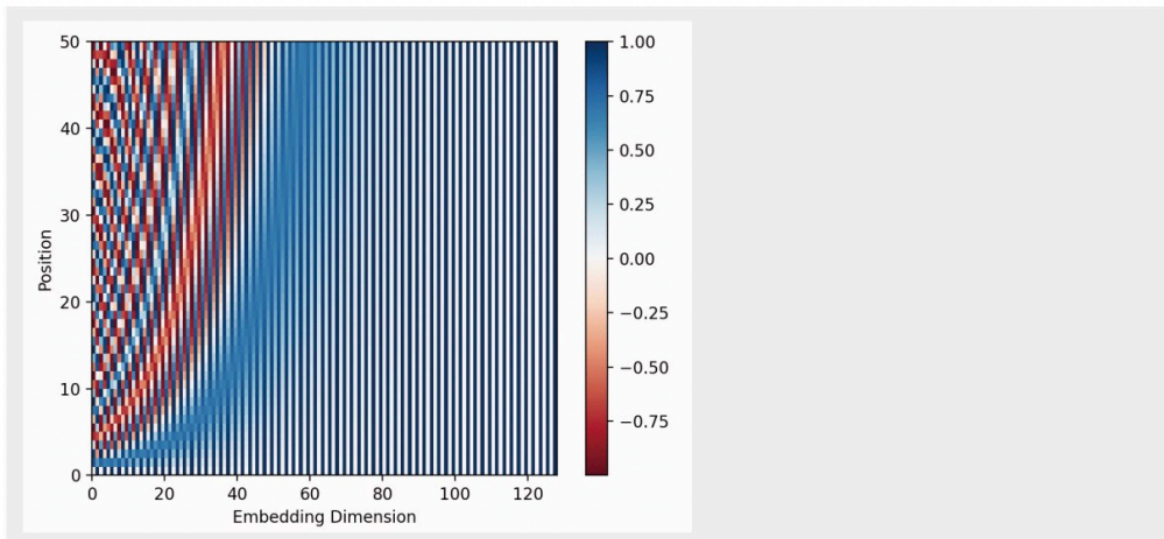
def forward(self, x):
    x = x + self.pe[: x.size(0)]
    return self.dropout(x)

encoding = PositionalEncoding(d_model=128, max_len=50)

plt.pcolormesh (encoding.pe.numpy().squeeze(), cmap="RdBu")
plt.xlabel("Embedding Dimension")
plt.xlim((0, 128))
plt.ylabel("Position")
plt.colorbar()
plt.show()

```

출력 결과



PositionalEncoding 클래스

- 임베딩 차원(d_model), 최대 시퀀스(max_len) 입력받음
- 입력 시퀀스의 위치마다 sin&cos 함수로 위치 인코딩 계산

- 위치 인코딩 계산 방식

수식 7.1 위치 인코딩

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

- pos: 입력 시퀀스에서 해당 단어의 위치
- i: 임베딩 벡터의 차원 인덱스
 - 짝수 → sin 함수 적용
 - 홀수 → cos 함수 적용
- pe 변수의 텐서 차원: [최대 시퀀스, 1, 입력 임베딩 차원]

특수 토큰

- 입력 시퀀스의 시작과 끝을 나타내거나 마스킹 영역으로 사용
- 모델이 입력 시퀀스의 시작과 끝을 인식 가능
- 마스킹으로 일부 입력 무시

특수 토큰을 활용한 문장 토큰 배열

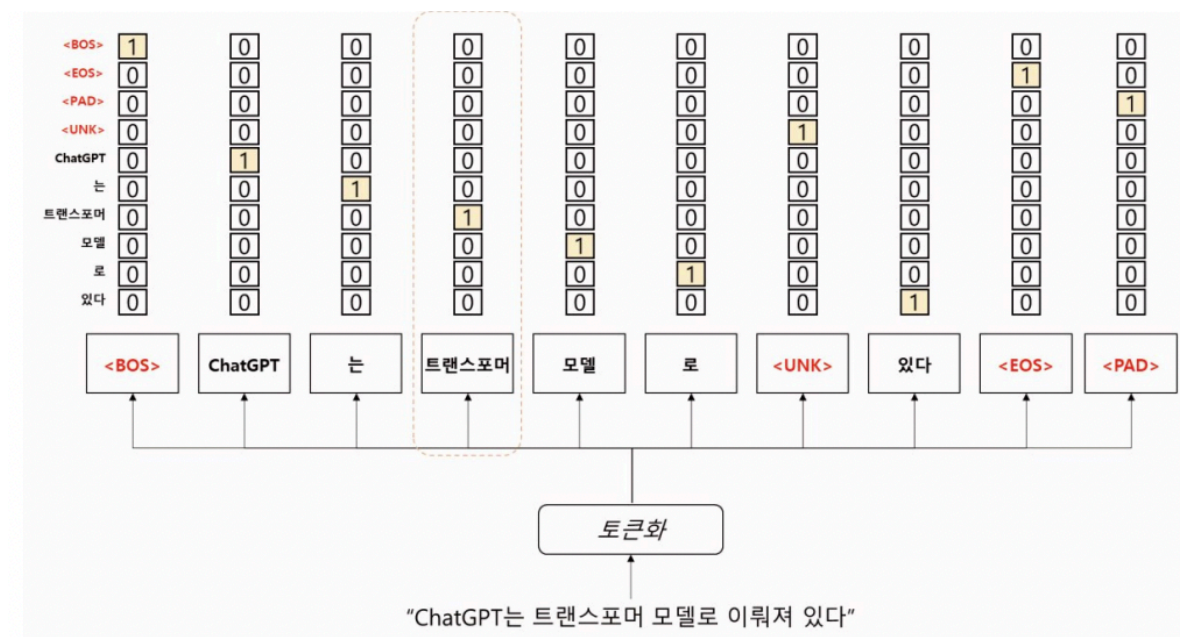


그림 7.3 특수 토큰을 활용한 문장 토큰 배열

- BOS: 문장의 시작
- EOS: 문장의 종료
- UNK: 어휘 사전에 없는 단어 (모르는 단어)
- PAD: 모든 문장을 일정한 길이로 맞추기 위해 사용 (짧은 문장의 빈 공간 채울 때)

→ 생성된 문장 토큰 배열을 어휘사전에 등장하는 위치에 원-핫 인코딩으로 표현

입력 임베딩 산출 과정

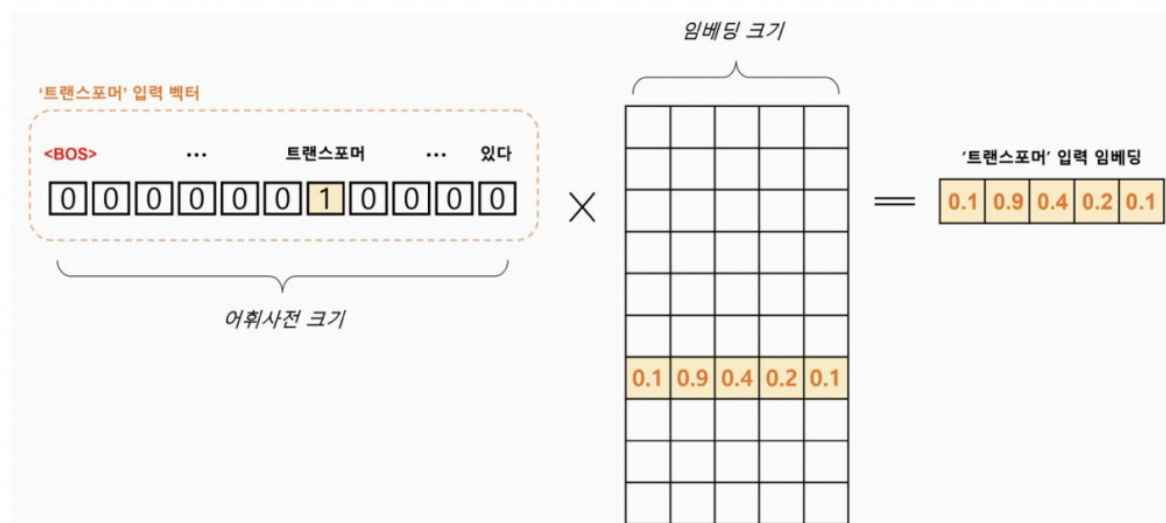


그림 7.4 입력 임베딩 산출 과정

- Word2Vec과 동일

트랜스포머 인코더

- 입력 시퀀스 받음 → 인코더 계층들을 거쳐 연산 수행
- 각 인코더 계층: 멀티 헤드 어텐션 + 순방향 신경망
 - 입력 데이터에 대한 정보 추출 후 다음 계층으로 전달
 - 위치 정보 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더함
 - 인코딩 계층 출력 → 디코더 계층으로 전달

트랜스포머 인코더 블록 연산과정

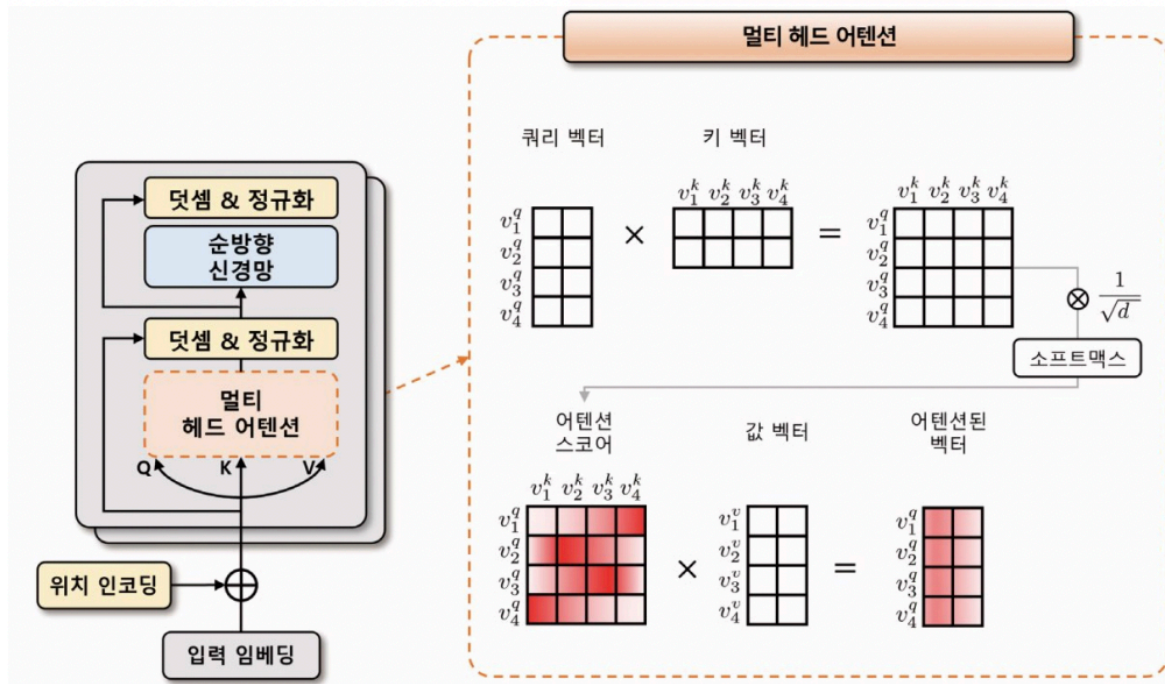


그림 7.5 트랜스포머 인코더 블록 연산 과정

- 인코더는 위치 인코딩이 적용된 입력 임베딩 받음
- 입력 텐서 [배치 크기 N, 시퀀스 길이 S, 임베딩 차원 d]

Q, K, V 벡터 생성

- Q: 현재 시점이 무엇을 알고 싶은지 나타내는 질문 벡터
- K: 쿼리가 참고할 정보가 어디 있는지 나타내는 벡터
- V: 실제 정보 벡터, 어텐션 비율 반영해 조합
- 학습을 통해 서로 다른 의미로 최적화

어텐션 스코어 계산

수식 7.2 어텐션 스코어 계산식

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

멀티 헤드 어텐션

- 셀프 어텐션을 k번 병렬 수행
- 각 헤드의 텐서 크기: $[N, S, d/k]$
- 전체 형태: $[N, k, S, d/k]$
- 각 헤드의 결과를 concat해서 다시 $[N, S, d]$ 로 합침

잔차 연결 & 계층 정규화

- 입력 + 어텐션 출력 → 잔차 연결
- → Layer Normalization 적용
- ⇒ 기울기 소실 완화 + 안정적 학습

순방향 신경망(FNN)

- 선형 변환 → ReLU → 선형 변환
OR
- 1D 합성곱 기반 FFN
→ 다시 잔차 연결 & 정규화

인코더

- 여러개의 동일한 블록이 쌓여 있음
- 앞 블록의 출력 → 다음 블록 입력
⇒ 점차 의미 추상화 & 고차원적 표현 형성
- 마지막 인코더 출력 벡터: 디코더에서 K, V로 사용
- 디코더가 이를 참조해 적절한 타겟 시퀀스 생성

트랜스포머 디코더

디코더 입력

- 위치 인코딩이 적용된 타겟 시퀀스 임베딩
- 위치 인코딩 → 타겟 문장의 단어 순서 정보 학습

마스크 멀티 헤드 어텐션

- 미래 단어 참조하지 못하도록 마스크 적용
- Q 위치에 따라 볼 수 있는 K의 범위 제한
- 목적: 미래 정보 참조 차단(인과성 보장)
- 참조하면 안되는 위치에 -무한대 값 더함 → softmax에서 확률이 0
→ 디코더는 현재 시점 이전의 단어 정보만으로 셀프 어텐션 수행

디코더 블록 구조

1. Masked Multi-Head Attention

- 타깃 시퀀스 내부에서 인과성 유지하며 셀프 어텐션 계산

2. Encoder-Decoder Attention

- 디코더의 Q가 인코더 출력의 K, V값 참조
- 번역에서 입력 문장의 의미 활용하는 단계

3. Feed-Forward Network

- 비선형 변환으로 의미 정제&확장

최종 역할

- 디코더: 이전에 생성된 단어만 보고 다음 단어 생성
- 인코더의 정보를 함께 참고 → 더 정확한 단어 선택
- 여러 디코더 블록 거쳐 표현 고도화
- 마지막 출력은 선형 변환 & softmax 거쳐 실제 단어로 변환

트랜스포머 디코더 블록 연산 과정

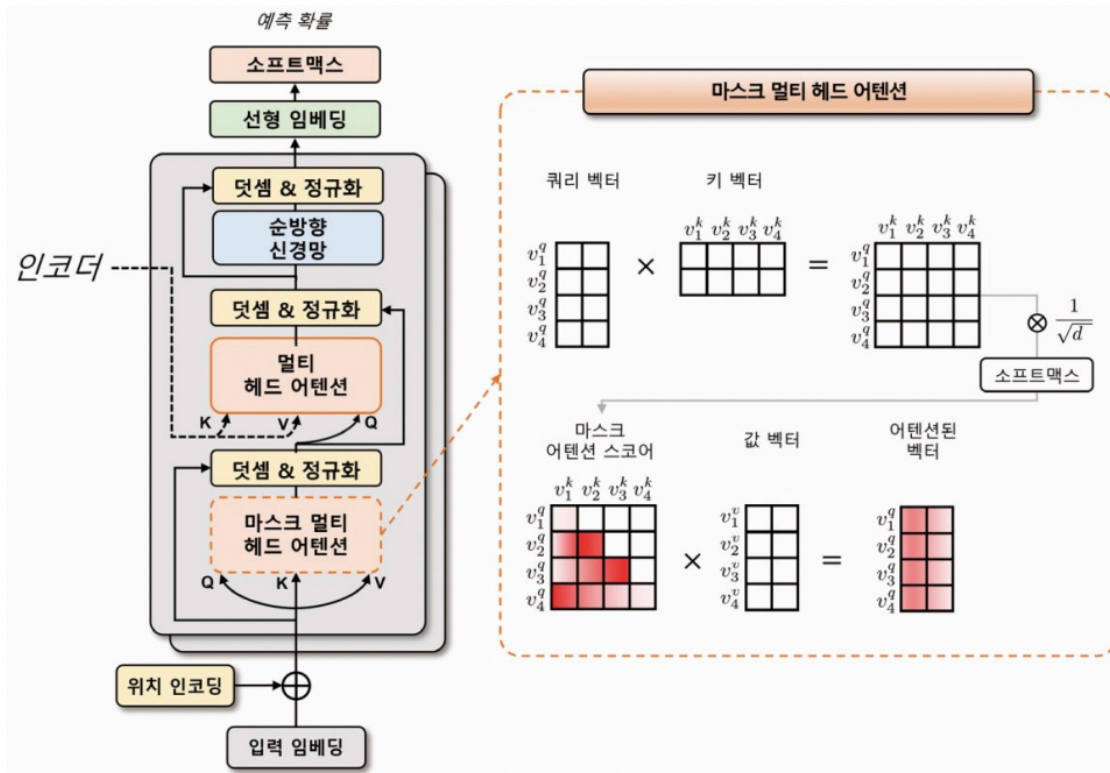


그림 7.6 트랜스포머 디코더 블록 연산 과정

디코더 Q, K, V 구성

- Q: 타겟 시퀀스의 임베딩 + 위치 인코딩으로 생성
- K와 V는 인코더에서 나온 소스 시퀀스의 출력 사용
⇒ 타겟이 질문(Q)이 되고, 인코더가 가지고 있는 정보(K, V) 참고

어텐션 스코어 계산

1. Q와 K로 어텐션 스코어 맵 계산
2. softmax로 스코어 확률화
3. 확률 가중치로 V 가중합
⇒ 멀티 헤드 어텐션의 출력 벡터

마스킹 적용

- 마스크된 위치는 softmax에서 0이 돼서 영향이 사라짐
→ 디코더는 항상 현재보다 이전의 토큰 정보만 이용

디코더 블록 구성

- 여러 개의 디코더 블록 쌓아 구성
- 이전 디코더 블록의 출력은 → 다음 블록의 입력으로 전달
⇒ 시점이 지나며 표현이 점점 정교 & 더 정확한 타깃 예측

최종 출력 계산

- 마지막 디코더 블록의 출력 텐서 $[N, S, d]$ 에
선형 변환 + softmax 적용 → 각 위치별 다음 단어의 예측 확률
- 확률 바탕으로 다음 토큰 순차적 생성

PAD 토큰 활용

- 아직 생성되지 않은 위치에 PAD 토큰 채워 넣음

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

모델 실습

GPT(Generative Pre-trained Transformer)

- 대규모 말뭉치로 사전 학습된 모델로 다양한 다운스트림 작업에서 우수한 성능
- 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델
- 트랜스포머의 인코더: 입력 문장의 특징 추출
트랜스포머의 디코더: 입력 문장과 출력 문장 간의 관계 이해, 출력 문장 생성
- 디코더 쌓아 모델 구성 → 언어 모델링 작업에서 높은 성능
- 대규모 텍스트 데이터셋으로 사전 학습해 초기화, 특정 자연어 처리 작업에 맞게 미세 조정해 사용
- 자연어 처리 작업에 활용, 예측 성능 뛰어남, 적은 데이터로도 높은 성능
- 다양한 시리즈 - 거의 동일한 트랜스포머 구조, 점점 더 많은 트랜스포머 계층과 데이터
- GPT1: 매개변수 1억 2천만개
GPT2: 매개변수 15억개
GPT3: 매개변수 1750개 초거대 모델 (많은 자원 필요)
→ 많은 양의 매개변수를 줄이기 위해
GPT3.5: RLHF 도입
 - RLHF: 인간의 피드백을 이용하는 강화 학습 방법
 - 인간이 모델이 생성한 결과물 평가, 모델 보상
- GPT3.5: ChatGPT 서비스 공개
GPT4: 이미지 데이터 함께 학습
- 그외: GPT-Neo, GPT-J

GPT-1

- 트랜스포머 구조를 바탕으로 한 단방향(Unidirectional) 언어 모델
 - 단방향: 텍스트 생성시 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어 예측
 - 이전 단어들을 차례대로 입력하면서 다음 단어 예측
→ 모델의 계상량 감소 → 빠른 학습 속도, 모델 크기 작음
- 트랜스포머 구조 중 디코더 부분만 사용, 인코더-디코더 간 어텐션 연산 제외
→ 모델의 매개변수 줄이면서도 높은 성능

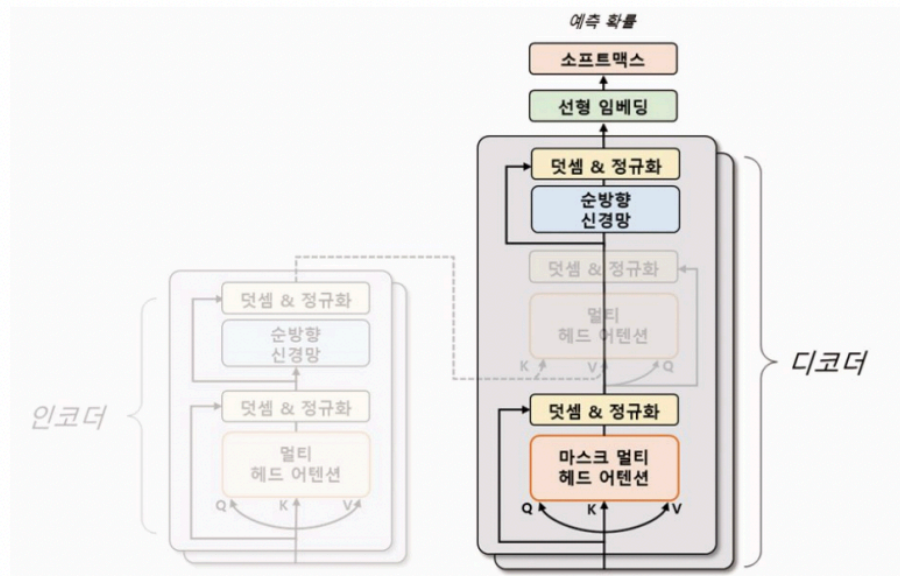


그림 7.7 트랜스포머와 GPT-1 모델 비교

- 인코더 X, 디코더 부분만 사용
- 디코더의 두번째 다중헤드 어텐션 사용X
- 헤드 12개 - 디코더 12층 - 매개변수 1억 2천만 개
- 4.5GB의 BookCorpus 데이터셋으로 언어 모델 사전 학습
 - 입력 문장을 임의의 위치까지 보여주고, 뒤에 이어지는 문장을 모델이 자동 예측
 - 레이블 필요 X 비지도 학습
 - 장점: 자원이 충분하다면 제약 없이 학습 가능

GPT-1 사전 학습 작업 구조

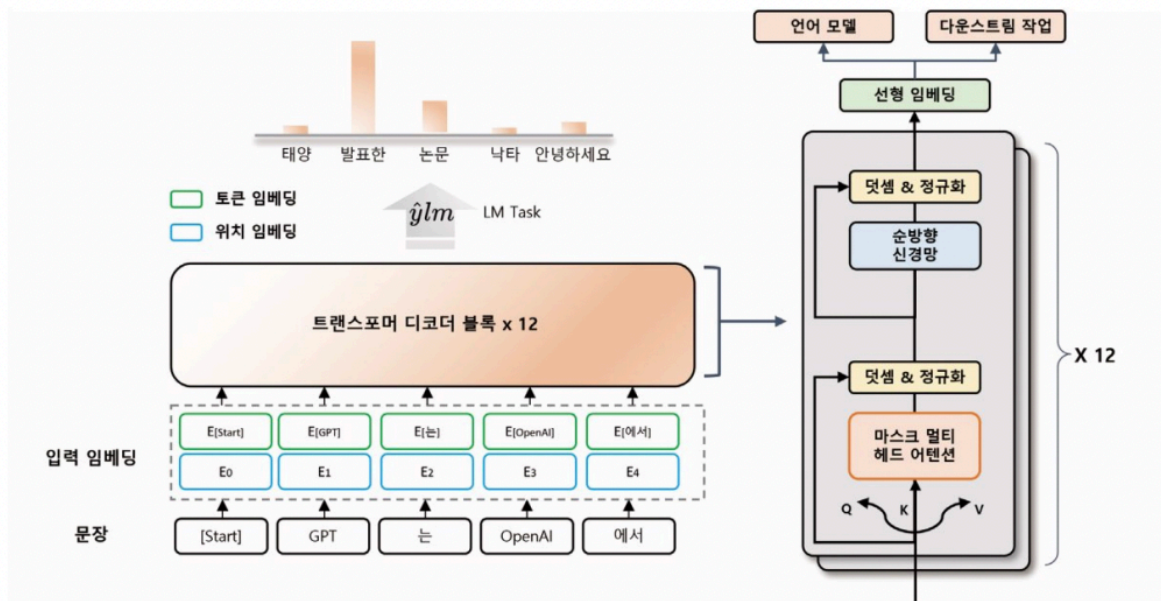


그림 7.8 GPT-1의 사전 학습 작업 구조

- 입력 문장을 일정한 길이의 블록으로 나누어 처리
- 각 블록에서 블록 내의 첫 번째 토큰부터 순서대로 다음 토큰을 예측하게 함
→ 장기적인 문맥 이해할 수 있도록

입력 임베딩 구조

- 토큰 임베딩 + 위치 임베딩 → 최종 입력 임베딩
- 토큰 임베딩: 각 토큰을 고정 차원의 벡터로 매핑
- 위치 임베딩: 문장에서의 토큰 위치 정보를 벡터로 표현

다운스트림 작업

- GPT-1은 다운스트림 학습에서도 입출력 구조 바꾸지 X
- 다양한 작업 처리할 때 모델 아키텍처 수정 X, 동일한 언어모델 형태로 미세조정
→ 구조 변경 없이 여러 NLP 작업에 적용 가능

보조학습(Auxiliary Learning)

- 미세 조정 시 보조학습 함께 사용
- 주요 학습 외에 작은 별도의 학습 작업을 동시에 수행

- 보조 작업에서 얻은 정보가 주요 작업 성능 향상에 기여

GPT-1의 다운스트림 작업 구조

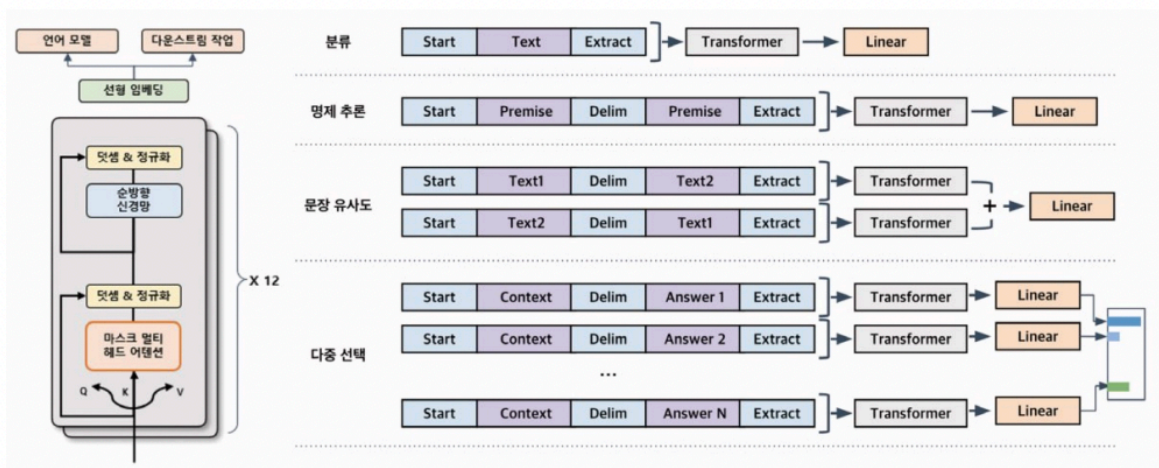


그림 7.9 GPT-1의 다운스트림 작업 구조

- 다운스트림 작업 - 원하는 목표에 맞게 모델 세밀 조정 필요
 - 일반적인 언어 모델 학습: 추가 손실 함수 필요
 - 두개의 손실 함수 사용해 최적화 → 보조학습을 통해 언어 모델 개선 & 다운스트림 작업 목표 달성 위한 정보 학습
 - 특수 토큰 사용: <start>, <delim>, <extract>

GPT-2

- 입력 문장 길이: 512 → 1024
- 디코더 층: 12 → 48 → 더욱 복잡한 패턴 학습 가능
- 매개변수: 1200만개 → 15억개

모델 구조

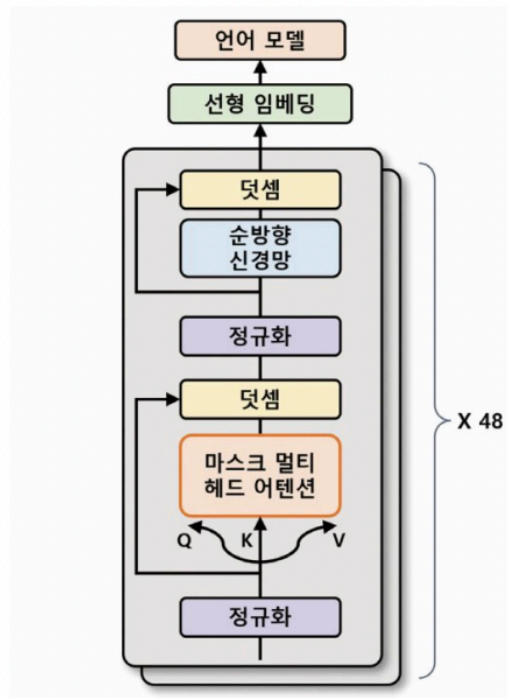


그림 7.10 GPT-2의 모델 구조

- 데이터: 미국의 웹사이트에서 스크래핑한 800만개 문서 40GB
- 더 자연스러운 문장 생성 능력
- 제로-샷 학습
 - 별도의 미세 조정 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습과정에서 특정한 형식의 데이터 입력
 - 'GPT는 놀라워요 = GPT is amazing'

GPT-3

- 모델 구조 자체는 거의 동일
- 매개변수 규모가 약 116배 증가. 초대형 모델
- 멀티 헤드 어텐션 헤드: 96개
- 트랜스포머 디코더: 96층
- 토큰 임베딩 차원: 1600 → 12,888
- 입력 최대 토큰 : 1024 → 2048
→ 더 긴 문장, 더 복잡한 문맥 처리 가능

- 어텐션 방식 변화: 일반 어텐션 + 희소 어텐션 혼합
→ 대규모 모델에서도 계산 효율 확보
- 다양한 출처에서 45TB 방대한 데이터셋 학습
- 다운스트림: 프롬프트 형식으로 작업 수행, 사용자가 주는 입력 문맥만으로 문제 해결
 - Few-shot
 - One-shot
 - Zero-shot

다운스트림 작업 프롬프트 예시

질의 응답 입력 프롬프트 문서 : 생성적 사전학습 트랜스포머(Generative Pre-Trained Transformer 3), GPT-3는 OpenAI에서 만든 대형 언어 모델이다. 질문 : GPT-3를 만든 곳은 어디인가? 답변 : OpenAI 출력값	자연어 추론 입력 프롬프트 자연어 추론 : OpenAI는 미국의 인공지능 연구소로, 인류에게 이익을 주는 것을 목표로 한다. GPT-3등 거대 언어 모델을 개발하여 많은 이목을 끌었다. 질문 : OpenAI는 상장사인가? 참, 거짓, 둘 다 아님 답변 : 둘 다 아님 출력값
수학 문제 풀이 입력 프롬프트 질문 : $(2 * 4) * 6$은 무엇인가? 답변 : 48 출력값	일반 상식 질의 응답 입력 프롬프트 질문 : 물은 섭씨 몇 도에서 어는가? 답변 : 0도 출력값

그림 7.11 GPT-3의 다운스트림 작업 프롬프트 예시

장점

- 높은 일반화 능력, 새로운 도메인의 작업도 잘 수행
- NLP 분야에서 이전의 기술적 한계 극복

한계

- 모델 크기가 너무 큼 → 추론 속도 느림, 비용 많이 듦

- 사전 학습 데이터에 강하게 의존 → 데이터 편향/부정확한 학습 내용
- 인간 수준의 이해력, 상식 기반 추론 능력 부족
- 편견, 오류, 혐오 표현 포함 가능성
- 산출물에 대한 저작권 문제

GPT-3.5(InstructGPT)

- RLHF 도입 → 매개변수 수 줄임, 자연스러움 높임

RLHF 방법

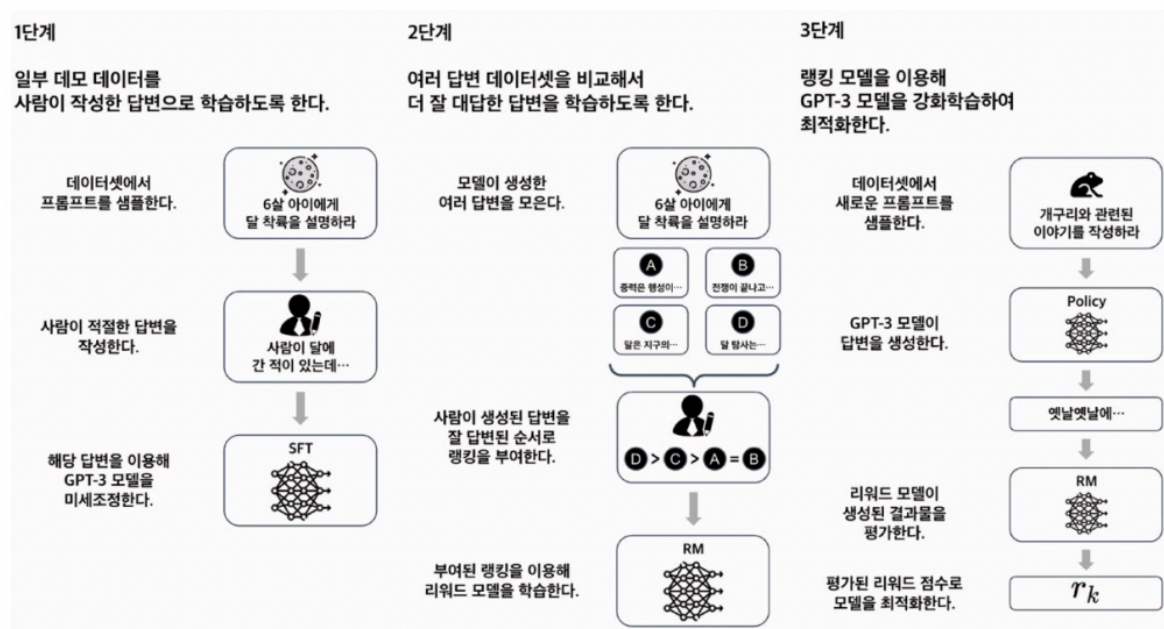


그림 7.12 RLHF GPT-3 학습 방법

1. 데이터셋에서 임의의 프롬프트 선택
2. 사람이 해당 프롬프트에 대해 직접 정답 작성
3. 사람의 답변과 GPT 모델이 생성한 답변을 비교
 - 자연스러우면 보상 제공
 - 부자연스러우면 보상 X
 - 보상 흐름을 바탕으로 강화학습 수행
 - 지도 미세 조정 과정

보상 모델(Reward Model)

- 모든 프롬프트에 사람이 일일이 답변 불가능
→ 사람의 평가를 학습한 보상 모델
- GPT는 seed에 따라 서로 다른 답변 여러개 생성 가능
→ 여러 후보 답변 생성한 뒤 사람이 랭킹 매김
→ 랭킹 정보를 이용해 보상 모델 학습
→ 이후에는 사람이 직접 평가하지 않아도 보상 모델이 자동으로 품질 평가

RLHF 학습 구조

- state(상태) = 프롬프트
 - policy(정책) = 모델이 생성하는 답변
 - action(행동) = 생성된 문장
 - reward(보상) = 보상 모델이 평가한 점수
-
- RLHF 반복 학습
→ 더 자연스럽고 인간 의도에 적합한 문장을 생성하는 모델
 - 한계
 - 환각(Hallucination)
 - 모델이 실제 이해/판단 능력 X, 학습 데이터의 패턴을 기반으로 답 생성

GPT-4

- 멀티모달 모델: 이미지도 입력으로 처리 가능
- 입력 토큰 수 확대: 4,096 → 32,768개 (8배 증가)
→ 긴 문서 분석, 대규모 문맥 이해 능력 향상
- 벤치마크 성능 향상
 - MMLU 성능
 - 미국 변호사 시험, SAT 읽기/쓰기
- 모델 세부 정보 비공개

모델 실습